Lập trình mạng · Chương 3

HTTP/3: QUIC và giao thức trên UDP

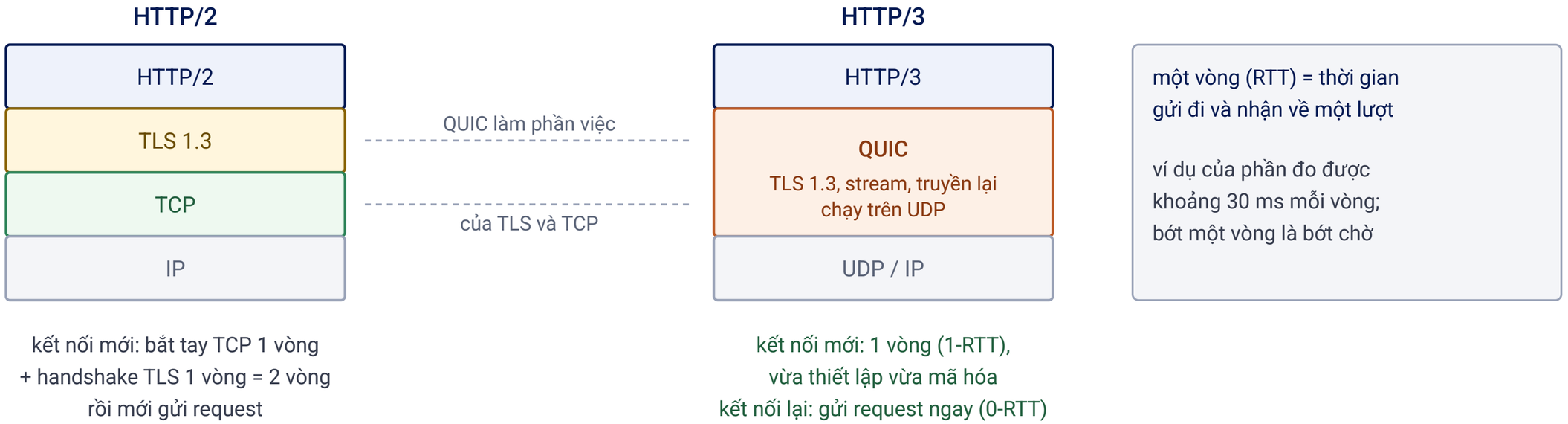
TS. Đặng Ngọc Hùng

Ba giới hạn nằm trong chính TCP

1. Mất một segment TCP: **mọi stream** trên kết nối HTTP/2 phải chờ
2. Kết nối mới: bắt tay TCP rồi handshake TLS, **2 vòng** trước request đầu tiên
3. Kết nối TCP gắn với 4-tuple: điện thoại **đổi mạng** thì kết nối mất

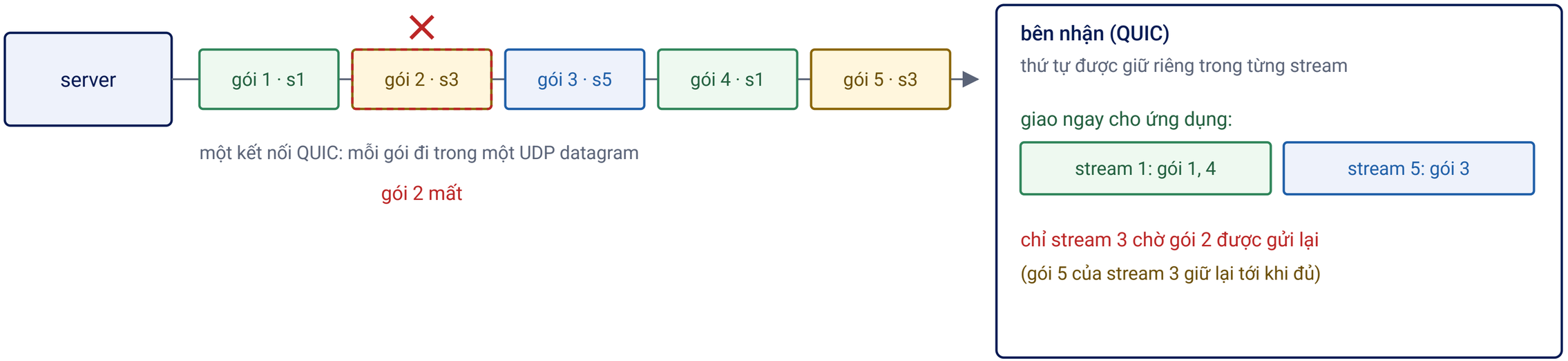
> **Câu hỏi của phần:** QUIC chạy trên UDP để vượt qua ba giới hạn này thế nào?

QUIC thay TLS và TCP, chạy trên UDP, thiết lập trong một vòng



QUIC: giao thức giao vận trên UDP, tự truyền lại, giữ thứ tự, mã hóa bằng TLS 1.3; kết nối mới **1-RTT**, kết nối lại **0-RTT**.

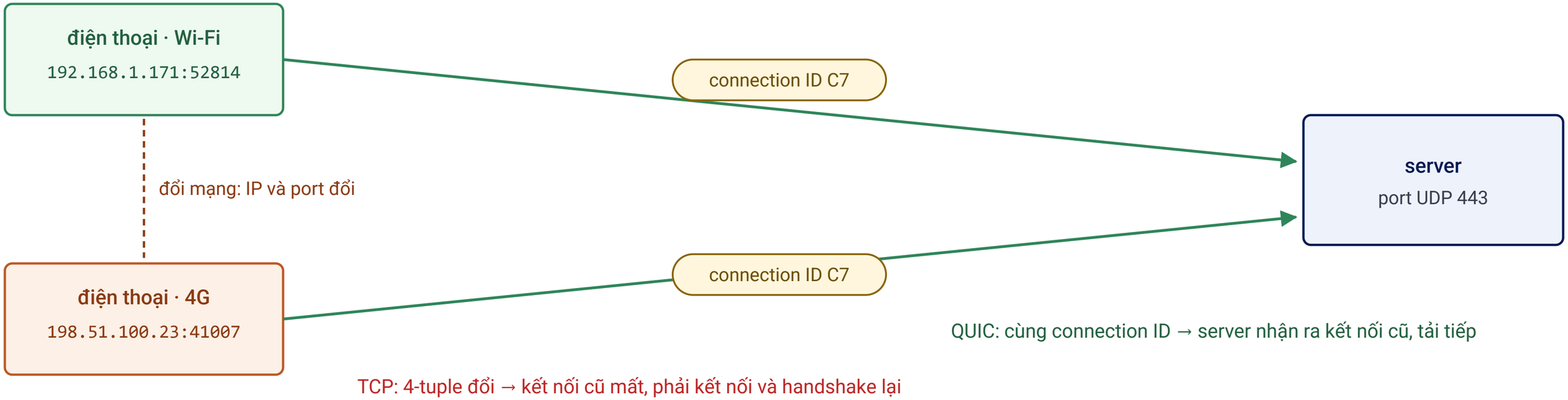
Stream độc lập: mất gói chỉ làm chậm stream chứa gói đó



mất một gói của stream 3 → chỉ stream 3 chờ: không còn head-of-line blocking giữa các stream

Stream độc lập: QUIC giữ thứ tự trong từng stream; mất gói chỉ làm chậm stream chứa gói đó.

Connection ID giữ kết nối khi điện thoại đổi mạng



Connection migration: kết nối QUIC định danh bằng **connection ID**, không bằng 4-tuple, nên giữ được khi đổi mạng.

Server báo có HTTP/3 bằng header Alt-Svc

Java

```
HttpClient client = HttpClient.newHttpClient();       // mặc định ưu tiên HTTP/2
HttpResponse<String> response = client.send(HttpRequest.newBuilder(uri).build(),
        HttpResponse.BodyHandlers.ofString());
System.out.printf("%s: status %d, version %s%n", uri, response.statusCode(), response.version());
System.out.println("alt-svc: " + response.headers().firstValue("alt-svc").orElse("(none)"));  // h3 = HTTP/3
```

Kết quả chạy

```
https://cloudflare-quic.com/: status 200, version HTTP_2
alt-svc: h3=":443"; ma=86400
```

Request HTTP/3 thật qua QUIC: 69 UDP datagram, QUIC tự gửi lại gói mất

Java

```
HttpClient client = Http3Client.newHttpClient();     // cùng API HttpClient, bên dưới là QUIC trên UDP
HttpResponse<String> response = client.send(HttpRequest.newBuilder(uri).build(),
        HttpResponse.BodyHandlers.ofString());
Statistics stats = ((Http3Client) client).getConnectionStatistics();
System.out.printf("QUIC: %d UDP datagrams sent, %d packets lost (resent by QUIC), smoothed RTT %d ms%n",
        stats.datagramsSent(), stats.lostPackets(), stats.smoothedRtt());
```

Kết quả chạy

```
https://cloudflare-quic.com/ over HTTP/3: status 200, 125905 bytes
QUIC: 69 UDP datagrams sent, 2 packets lost (resent by QUIC), smoothed RTT 29 ms
```

0-RTT có thể bị phát lại, và mạng chặn UDP thì quay về HTTP/2

1. **0-RTT**: dữ liệu ở gói đầu có thể bị **phát lại**; chỉ dùng cho request gửi lặp lại không đổi dữ liệu, như GET
2. Mạng chặn UDP port 443: client quay về **HTTP/2 trên TCP**

> **Lưu ý:** HTTP/3 là lựa chọn nhanh hơn khi có, không thay hẳn HTTP/2.

Câu hỏi 1

Vì sao QUIC chạy trên UDP thay vì chạy trực tiếp trên IP như TCP?

A. Vì thiết bị mạng trên đường đi và hệ điều hành chỉ quen TCP và UDP; chạy trên UDP, QUIC đi qua được mạng hiện có và cài được ngay trong ứng dụng.

B. Vì UDP tự truyền lại gói bị mất.

C. Vì UDP có sẵn mã hóa.

D. Vì IP không cho phép giao thức giao vận nào khác ngoài TCP.

Câu hỏi 1

Vì sao QUIC chạy trên UDP thay vì chạy trực tiếp trên IP như TCP?

A. Vì thiết bị mạng trên đường đi và hệ điều hành chỉ quen TCP và UDP; chạy trên UDP, QUIC đi qua được mạng hiện có và cài được ngay trong ứng dụng.

B. Vì UDP tự truyền lại gói bị mất.

C. Vì UDP có sẵn mã hóa.

D. Vì IP không cho phép giao thức giao vận nào khác ngoài TCP.

UDP không truyền lại, không mã hóa; QUIC tự làm các việc đó. QUIC dùng UDP như lớp vỏ để đi qua thiết bị mạng và hệ điều hành hiện có, và để cài đặt trong ứng dụng thay vì trong nhân hệ điều hành.

Câu hỏi 2

Client kết nối lại tới một server đã biết và có thể gửi dữ liệu bằng 0-RTT. Request nào an toàn để gửi bằng 0-RTT?

A. POST chuyển tiền.

B. POST đặt hàng.

C. GET đọc một trang tin.

D. POST gửi bình luận.

Câu hỏi 2

Client kết nối lại tới một server đã biết và có thể gửi dữ liệu bằng 0-RTT. Request nào an toàn để gửi bằng 0-RTT?

A. POST chuyển tiền.

B. POST đặt hàng.

C. GET đọc một trang tin.

D. POST gửi bình luận.

Dữ liệu 0-RTT có thể bị phát lại. GET đọc trang gửi lặp lại không thay đổi dữ liệu nên an toàn; ba request POST còn lại, nếu bị phát lại, sẽ chuyển tiền, đặt hàng hay đăng bình luận thêm lần nữa.

Tổng kết

1. **HTTP/3** chạy trên **QUIC**, QUIC chạy trên UDP: tự truyền lại, giữ thứ tự, mã hóa bằng TLS 1.3

2. Kết nối mới 1-RTT, kết nối lại **0-RTT** (chỉ cho request không đổi dữ liệu)

3. **Stream độc lập**: mất gói chỉ chậm stream đó; **connection migration** nhờ **connection ID**

Từ khóa cần nhớ QUIC · HTTP/3 · stream độc lập · 0-RTT · connection migration · connection ID

Mở rộng

Chủ đề nâng cao

- Thư viện QUIC cho Java (Netty incubator, Kwik)
- Kiểm soát tắc nghẽn trong QUIC

Bài tập mở rộng

Lập bảng so sánh HTTP/1.1, HTTP/2, HTTP/3 (tầng giao vận, số kết nối, head-of-line blocking, thời gian thiết lập); tìm một trang web dùng HTTP/3 và chứng minh bằng công cụ.

Tổng kết chương 3

1. **UDP**: datagram giữ ranh giới, không kết nối; tự đặt timeout, cấp buffer đủ lớn
2. **Multicast**: một lần gửi cho cả nhóm; tham gia nhóm trên đúng network interface; video đánh số thứ tự
3. **WebSocket**: nâng cấp từ HTTP, frame hai chiều; phòng chat với tập phiên an toàn đồng thời
4. **HTTP/2 → HTTP/3**: nhiều stream trên một kết nối TCP, rồi QUIC trên UDP với stream độc lập

Tiếp theo

RPC: lời gọi thủ tục từ xa và Java RMI